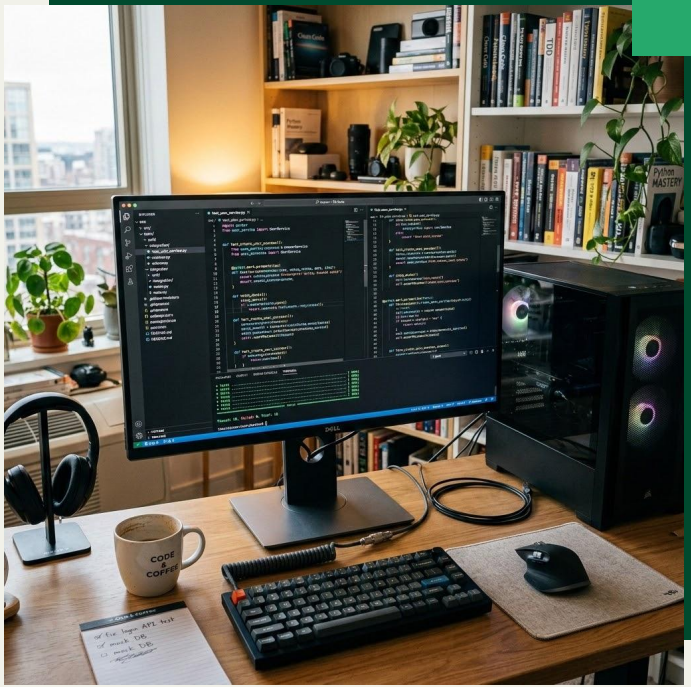
# Code Desensitizer for Generating Test Cases



**Pinardy Yang - May 2026**

# Contents

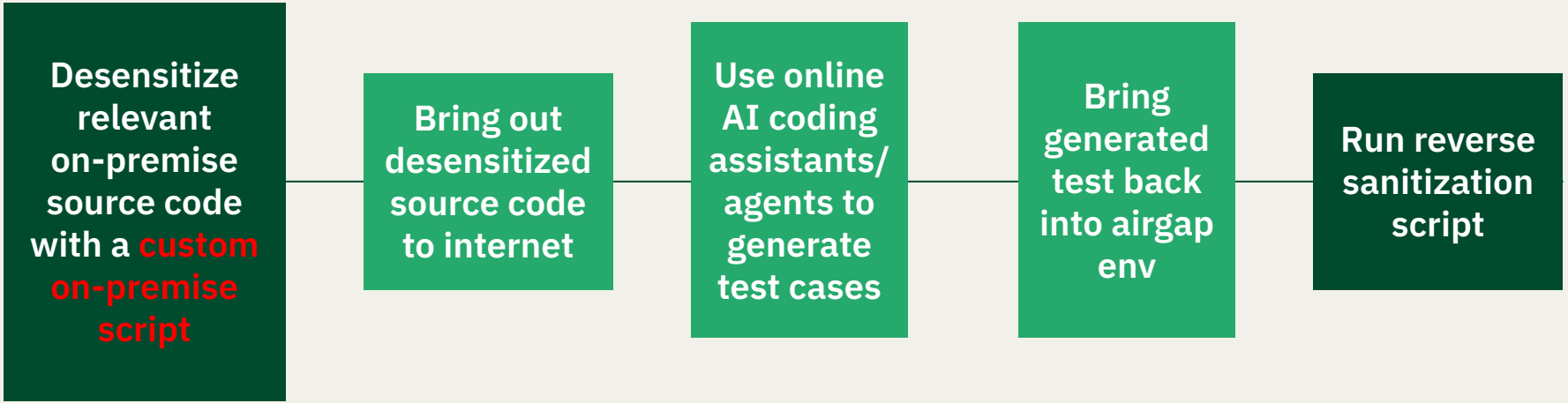
# Problem Statement

**There is poor performance when generating test cases on-premise due to existing infrastructure's performance**

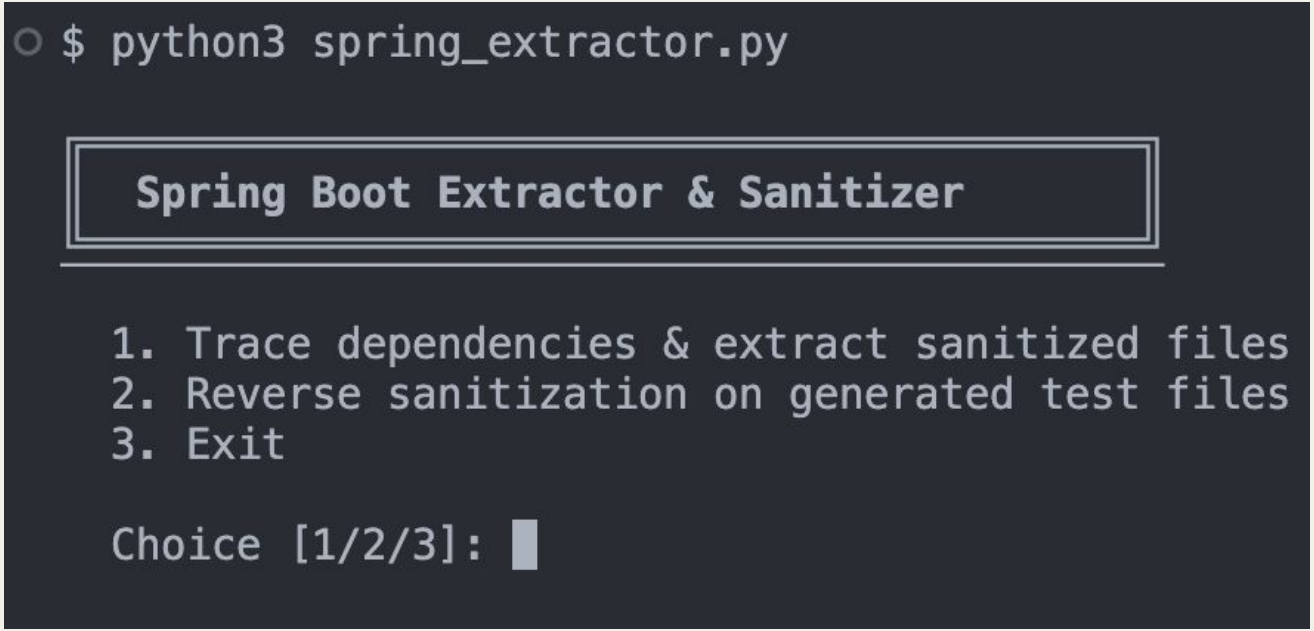
# Solution

**Utilize online AI coding assistants or agents to generate test cases**

# Workflow


Desensitize relevant on-premise source code with a custom on-premise script
Bring out desensitized source code to internet
Use online AI coding assistants/ agents to generate test cases
Bring generated test back into airgap env
Run reverse sanitization script

# Demo


$ python3 spring_extractor.py
Spring Boot Extractor & Sanitizer
1. Trace dependencies & extract sanitized files
2. Reverse sanitization on generated test files
3. Exit
Choice [1/2/3]:

```
$ python3 spring_extractor.py

  Spring Boot Extractor & Sanitizer

  1. Trace dependencies & extract sanitized files
  2. Reverse sanitization on generated test files
  3. Exit

  Choice [1/2/3]: 1

  Path to entry class (.java): /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/bac
kend/businessService/ingredient/IIngredientBusinessService.java
  Base package (e.g. com.mycompany.project): nus.iss.team3
  Source root [/Users/pinardy/Documents/Projects/feats/backend/src/main/java]:
  Output directory [./extracted]:
  Mapping file [extracted/mapping.json] (leave blank to define now):

Package / token mappings
  Enter sensitive→safe substitutions (e.g. com.classified → com.example)
  Press Enter with an empty 'from' to finish.

  Sensitive string (from): nus.iss.team3
  Safe replacement  (to) : abc.xyz.unclassified
    Mapped: nus.iss.team3 → abc.xyz.unclassified
  Sensitive string (from):
```

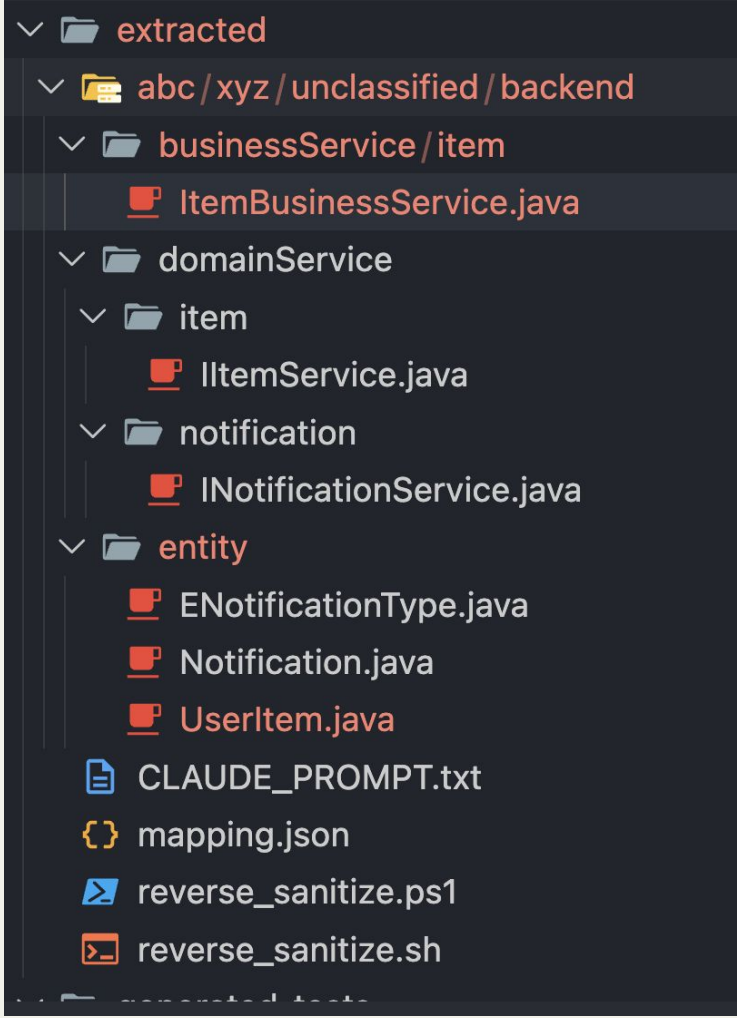
```
Variable/Class name mappings
  Rename classes and variables with automatic handling of variations
  E.g., 'Ingredient' → 'Item' will handle: Ingredient, ingredient, ingredients,
        IngredientService, INGREDIENT_TYPE, ingredient_service, etc.
  Press Enter with an empty 'from' to finish.

  Original name (from): Ingredient
  New name      (to) : Item
    Mapped: Ingredient → Item
      From variations: INGREDIENT, INGREDIENTS, Ingredient, Ingredients, ingredient...
  Original name (from):

Sanitization options
  Strip all comments [Y/n]:
  Remove @author / @since Javadoc tags [Y/n]:
  Mask string literals [y/N]:
  Remove logger statements [Y/n]:

Tracing dependencies...
  Found 6 class(es).
```

```
=== Extraction checklist ===

Services
  [ ] /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/backend/businessService/ingredient/IngredientBusinessService.java  ✓ found
  [ ] /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/backend/domainService/ingredient/IIngredientService.java  ✓ found
  [ ] /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/backend/domainService/notification/INotificationService.java  ✓ found

Domain models
  [ ] /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/backend/entity/ENotificationType.java  ✓ found
  [ ] /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/backend/entity/Notification.java  ✓ found
  [ ] /Users/pinardy/Documents/Projects/feats/backend/src/main/java/nus/iss/team3/backend/entity/UserIngredient.java  ✓ found

  Write sanitized files to output directory? [Y/n]:

Writing sanitized files...
  ✓ abc/xyz/unclassified/backend/businessService/item/ItemBusinessService.java
  ✓ abc/xyz/unclassified/backend/domainService/item/IItemService.java
  ✓ abc/xyz/unclassified/backend/domainService/notification/INotificationService.java
  ✓ abc/xyz/unclassified/backend/entity/ENotificationType.java
  ✓ abc/xyz/unclassified/backend/entity/Notification.java
  ✓ abc/xyz/unclassified/backend/entity/UserItem.java

  6 file(s) written to: extracted
  Prompt template → extracted/CLAUDE_PROMPT.txt
  Mappings saved → extracted/mapping.json
```

```
  6 file(s) written to: extracted
  Prompt template → extracted/CLAUDE_PROMPT.txt
  Mappings saved → extracted/mapping.json
  Bash reversal script  → extracted/reverse_sanitize.sh
  PowerShell reversal   → extracted/reverse_sanitize.ps1

═══ Done ═══
  Extracted files : extracted
  Mapping file    : extracted/mapping.json
  Claude prompt   : extracted/CLAUDE_PROMPT.txt

  Next steps:
  1. Paste each extracted file + CLAUDE_PROMPT.txt into Claude
  2. Save the generated tests to ./generated-tests/
  3. Run:  python spring_extractor.py reverse --mapping mapping.json --dir ./generated-tests
```

# End of sensitizing and extraction

Generated files from the sensitization & extraction script



mapping.json

extracted > {} mapping.json > ...

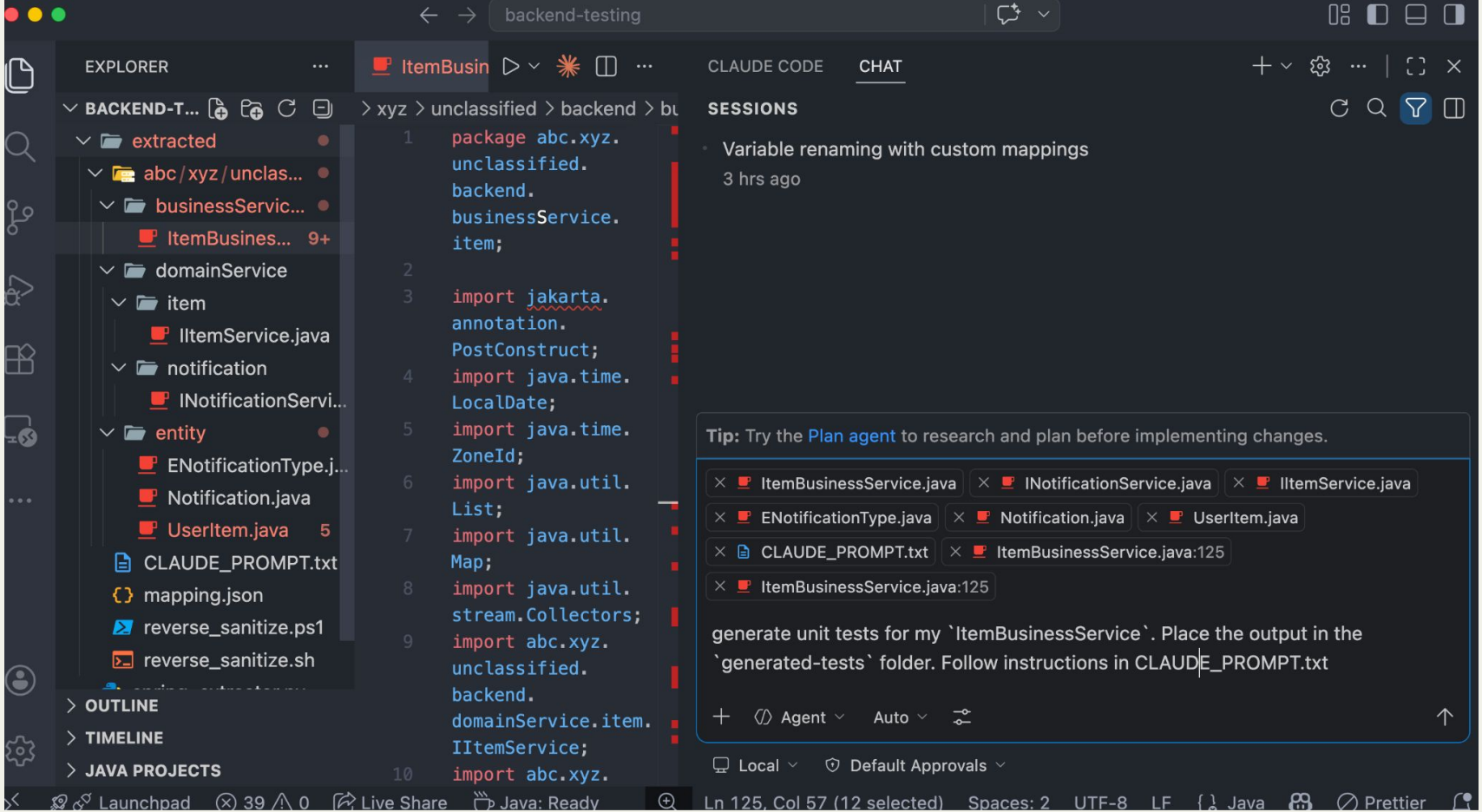
```json
{
  "package": [
    {
      "from": "nus.iss.team3",
      "to": "abc.xyz.unclassified"
    }
  ],
  "variable": [
    {
      "from": "Ingredient",
      "to": "Item"
    }
  ]
}
```

CLAUDE_PROMPT.txt

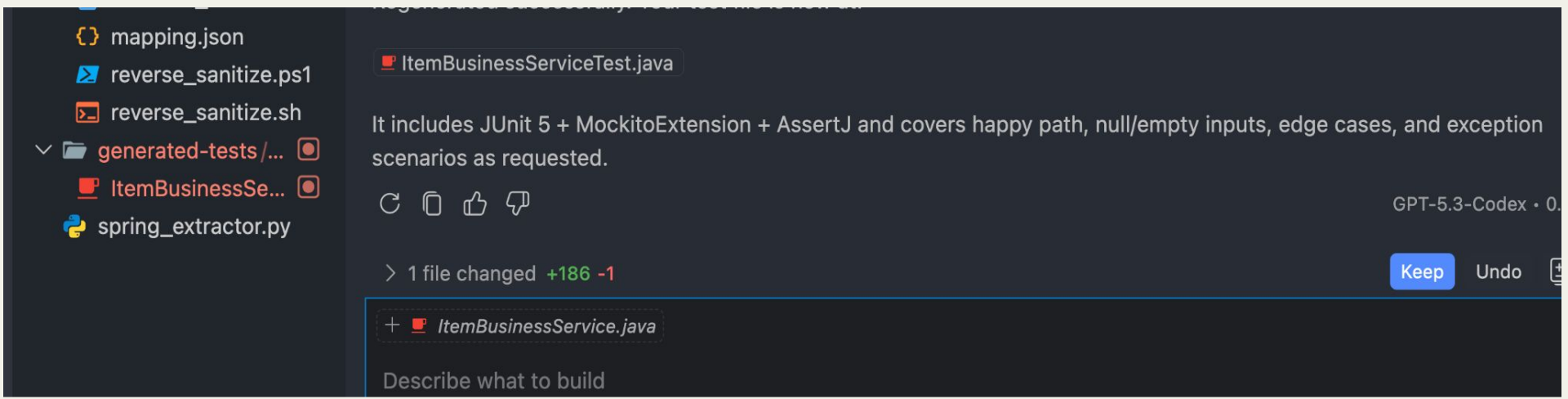
extracted > CLAUDE_PROMPT.txt

```
I have a Spring Boot service I need unit tests for.

Please generate comprehensive JUnit 5 unit tests using Mockito.

Requirements:
- Use @ExtendWith(MockitoExtension.class) — no @SpringBootTest
- Cover: happy path, edge cases, null/empty inputs, exception scenarios
- Use AssertJ assertions (assertThat)
- Mock all dependencies: IngredientBusinessService, IIngredientService,
INotificationService, ENotificationType, Notification, UserIngredient
- Do not assume any Spring context or infrastructure is running

Here are the sanitized source files:

[paste the contents of each extracted file below this line]
```

# Generated files from the sensitization & extraction script



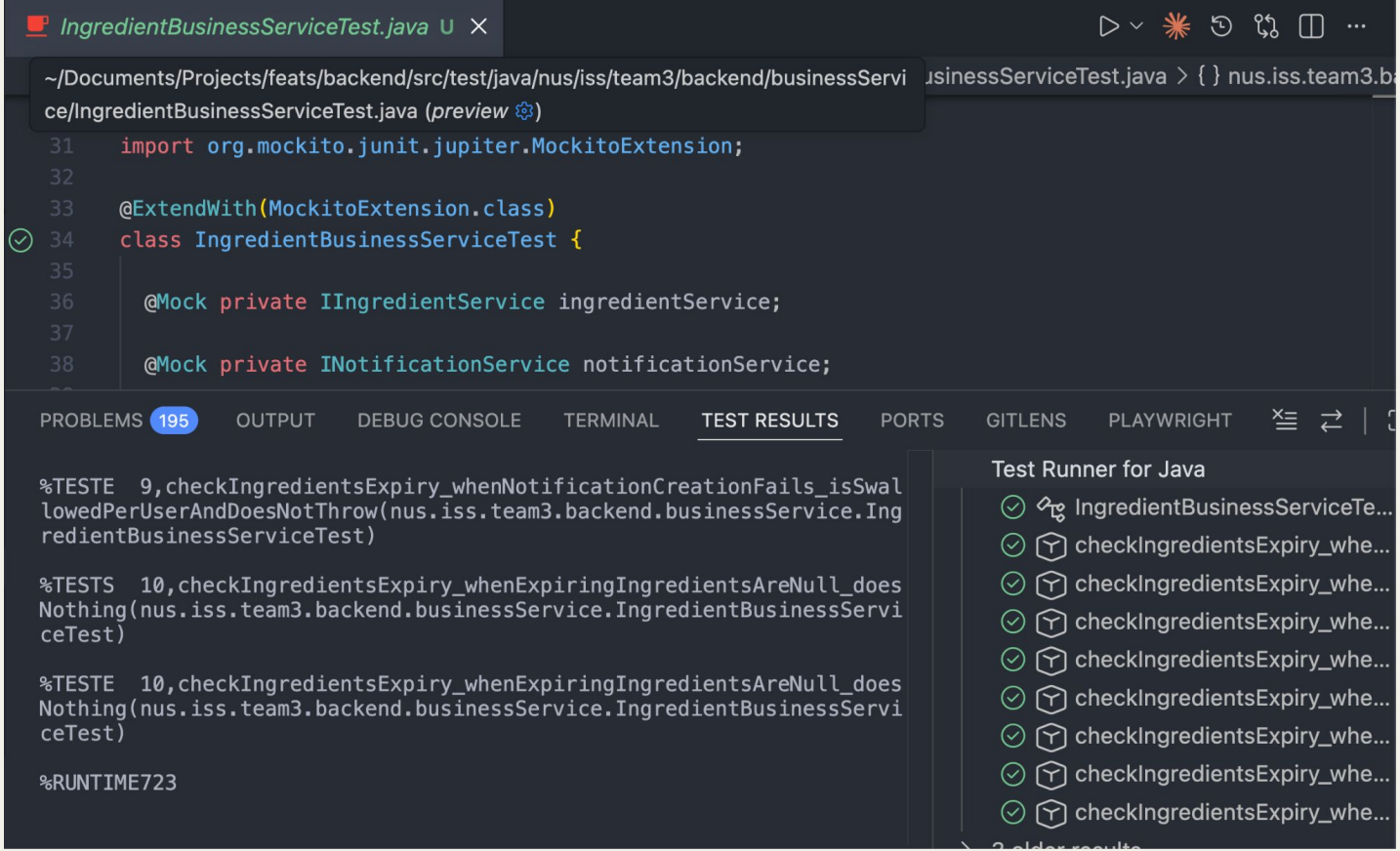

# Generate unit tests after running the desensitize script




# Generated unit tests with GitHub Copilot (you can use other tools too)

```
$ python3 spring_extractor.py

  Spring Boot Extractor & Sanitizer


  1. Trace dependencies & extract sanitized files
  2. Reverse sanitization on generated test files
  3. Exit

  Choice [1/2/3]: 2

  Path to mapping.json: /Users/pinardy/Documents/Projects/feats/backend-testing/extracted/mapping.json
  Directory containing generated test files: /Users/pinardy/Documents/Projects/feats/backend-testing/gene
rated-tests
  Loaded 1 package mapping(s) and 1 variable mapping(s)

Reversing mappings...
  Reversal complete — 1 file(s) renamed, 1/1 files updated.
```

# Run (2) - Reverse sanitization to convert from unclassified to confidential based on your `mapping.json`




Generated unit tests are passing
(after moving them back to project)

# About the script

- Single python script
- ~1000 lines of code
- Quite complex, not realistic to code this manually (too many edge cases)

# Development Process

I hit Claude's free tier limit here 🤡

Use various AI tools instead of just one


Entire process took **half a day** from start to completion

# Prompted with Claude

i have a spring boot project that is in a confidential airgapped environment. i am having trouble to generate unit test cases due to the lack of infrastructure to support AI agents. I want to bring out the relevant codes for my business logic out to the internet to generate unit tests before bringing it back into my airgapped environment. how should i approach this?

Here's a practical approach to safely extract code for AI-assisted test generation and bring it back:

### Strategy: Minimal Surface Area Extraction

The goal is to bring out **only what's needed** for unit test generation — no credentials, no infra config, no secrets.

---

### Step 1: Identify What to Extract

For each class you want tested, you need:

- **The target class itself** (service, util, doma gic)
- **Its direct dependencies** (interfaces/abstractions it uses — not implementations)

Write a message...

Sonnet 4.6

i would like a python script

Viewed a file, created a file, read a file >

**Requirements:** Python 3.7+ only — no pip installs needed.

---

**How to use it**

**Interactive mode** (easiest — just run it and follow the prompts):

```bash
python spring_extractor.py
```

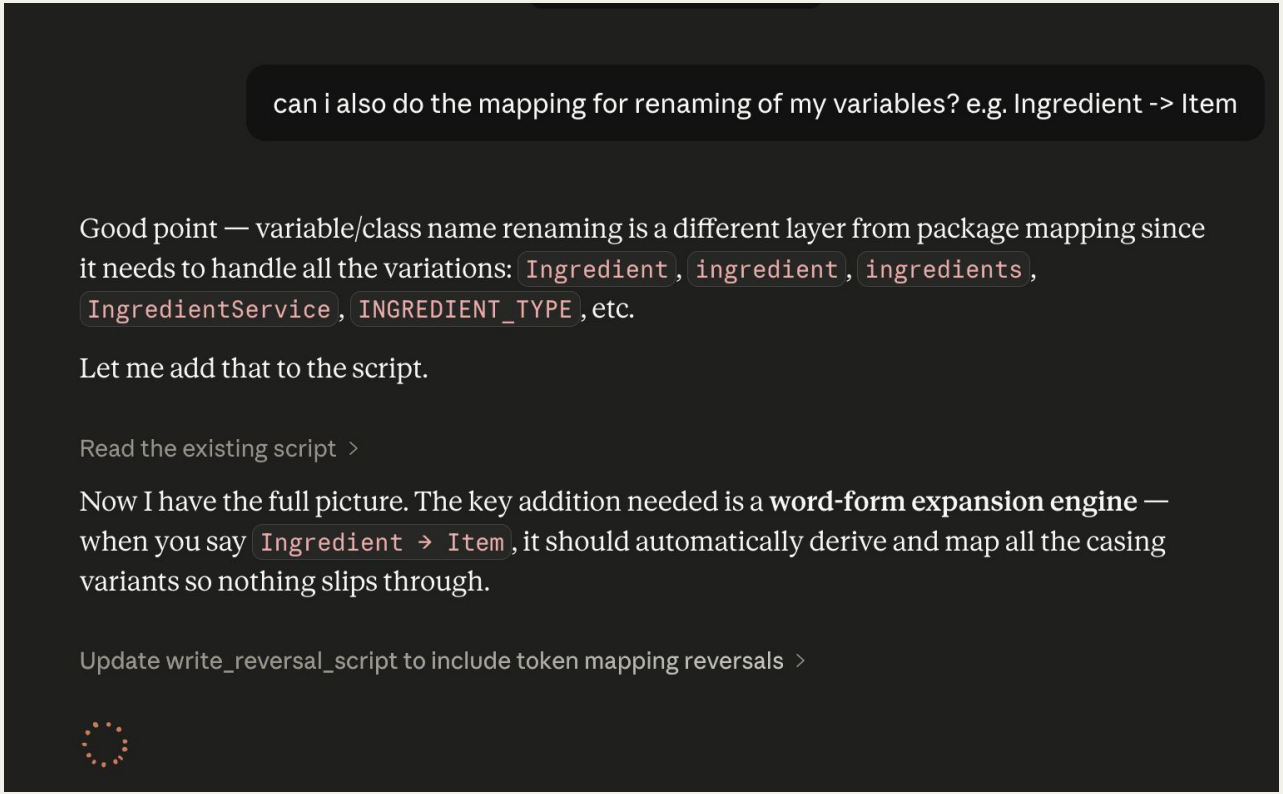
**CLI mode** (good for scripting):

```bash
# Step 1: Trace & extract
python spring_extractor.py trace \
  --entry src/main/java/com/myco/order/OrderService.java \
```

Write a message...

Sonnet 4.6

# Guide the prompt towards my intended goal




# Further refinements

CLAUDE CODE CHAT

VARIABLE RENAMING WITH CUSTOM MAPPINGS

keys.

### Automatic Variation Generation

From a single mapping like `Ingredient → Item`, the system generates and applies all variations:

| Pattern | Source | Target |
|---|---|---|
| PascalCase | `Ingredient` | `Item` |
| camelCase | `ingredient` | `item` |
| UPPER_CASE | `INGREDIENT` | `ITEM` |
| snake_case | `ingredient` | `item` |
| Plurals | `ingredients` | items |
| Compounds | `IngredientService` | `ItemService` |
| Constants | `INGREDIENT_TYPE` | `ITEM_TYPE` |

Todos (5/5)

1 file changed +310 -42 Keep Undo

spring_extractor.py

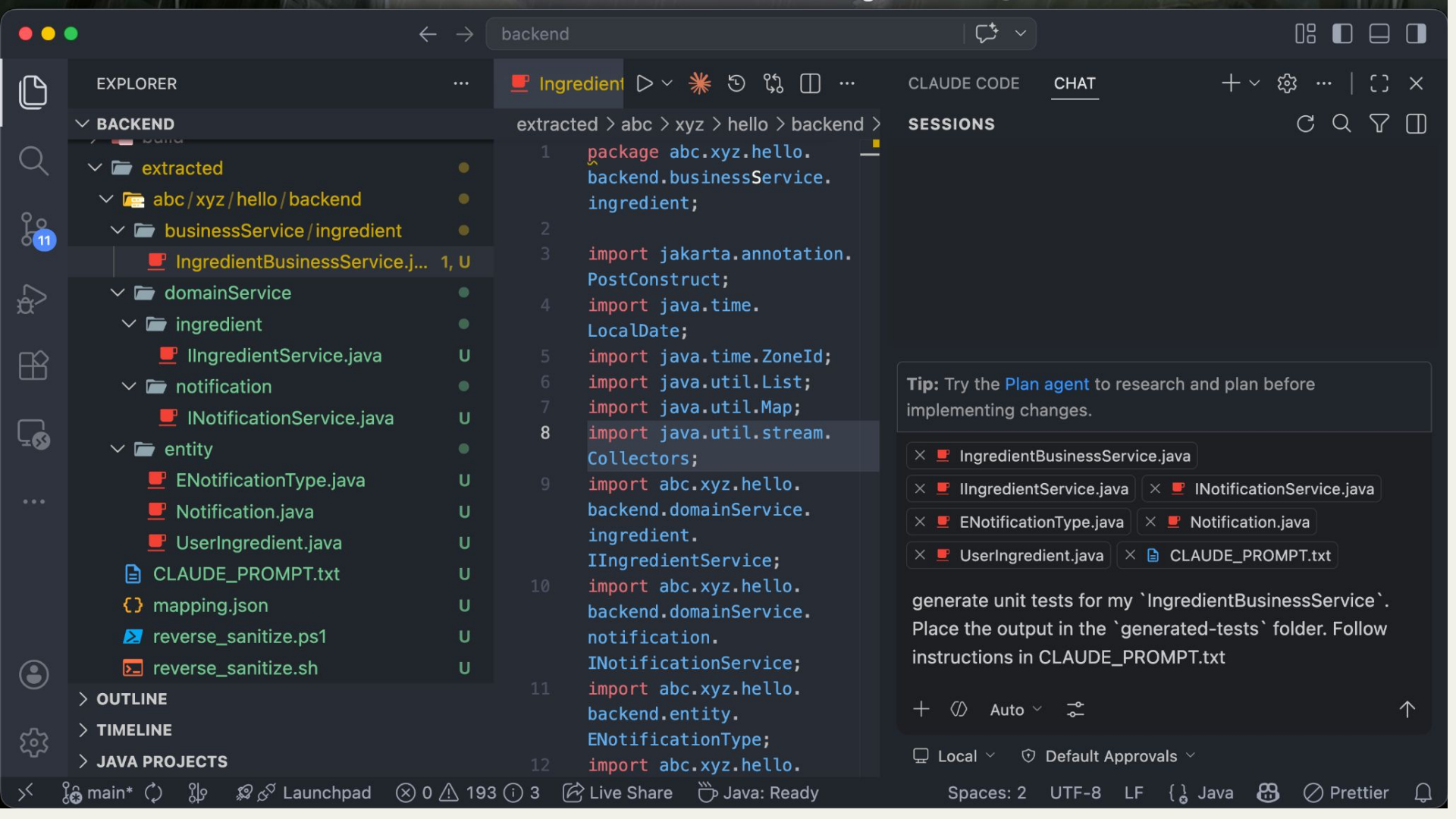
Describe what to build

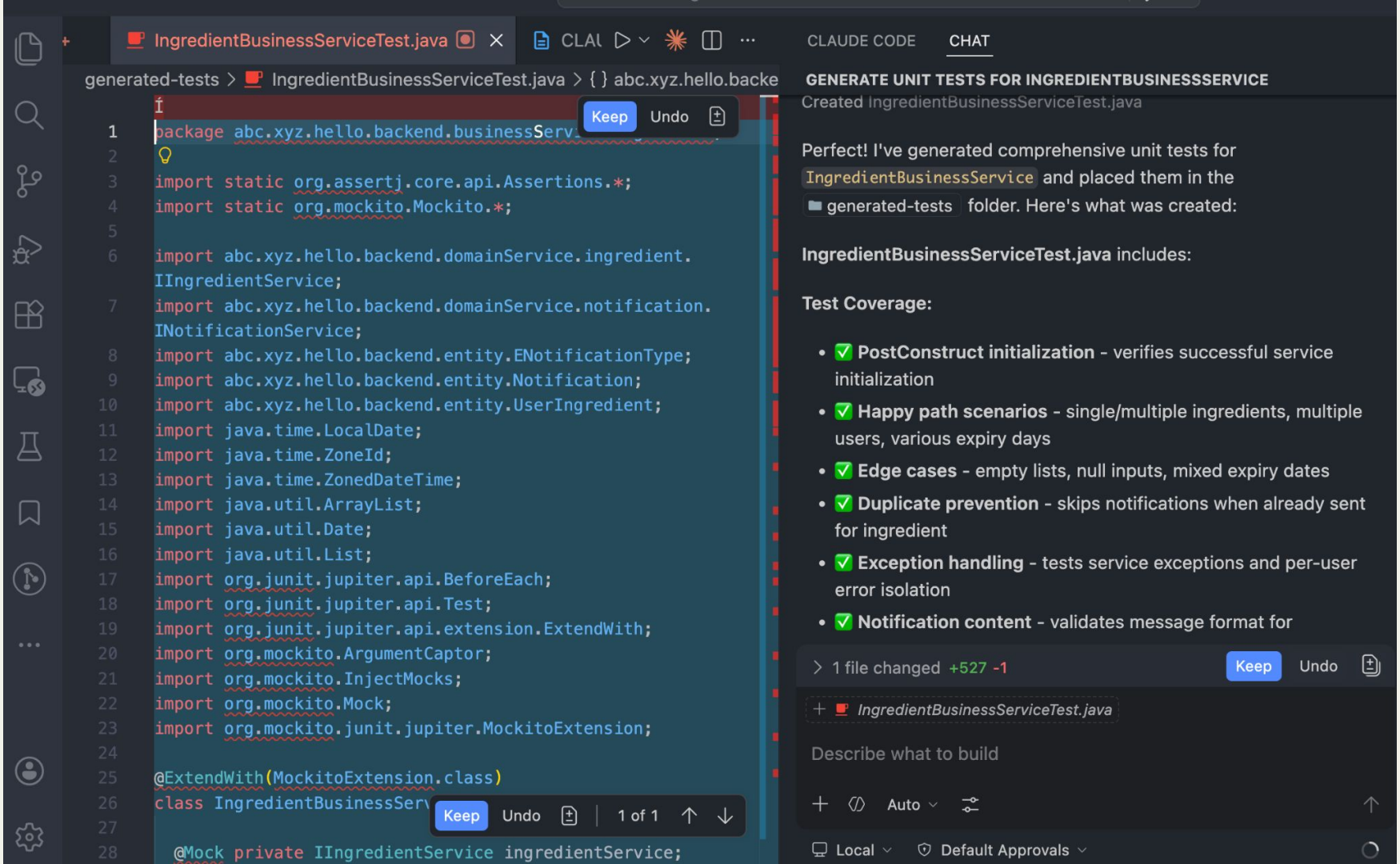
Agent Auto

Further refinements with:

GitHub Copilot

While I have a GitHub Pro subscription, my usage was also being throttled due to high usage


backend
EXPLORER
BACKEND
extracted
abc / xyz / hello / backend
businessService / ingredient
IngredientBusinessService.j... 1, U
domainService
ingredient
IIngredientService.java U
notification
INotificationService.java U
entity
ENotificationType.java U
Notification.java U
UserIngredient.java U
CLAUDE_PROMPT.txt U
mapping.json U
reverse_sanitize.ps1 U
reverse_sanitize.sh U
OUTLINE
TIMELINE
JAVA PROJECTS
Ingredient
extracted > abc > xyz > hello > backend >
1 package abc.xyz.hello.backend.businessService.ingredient;
2
3 import jakarta.annotation.PostConstruct;
4 import java.time.LocalDate;
5 import java.time.ZoneId;
6 import java.util.List;
7 import java.util.Map;
8 import java.util.stream.Collectors;
9 import abc.xyz.hello.backend.domainService.ingredient.IIngredientService;
10 import abc.xyz.hello.backend.domainService.notification.INotificationService;
11 import abc.xyz.hello.backend.entity.ENotificationType;
12 import abc.xyz.hello.
CLAUDE CODE
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
IngredientBusinessService.java
IIngredientService.java
INotificationService.java
ENotificationType.java
Notification.java
UserIngredient.java
CLAUDE_PROMPT.txt
generate unit tests for my `IngredientBusinessService`. Place the output in the `generated-tests` folder. Follow instructions in CLAUDE_PROMPT.txt
Auto
Local
Default Approvals
main*
Launchpad
0 193 3
Live Share
Java: Ready
Spaces: 2
UTF-8
LF
Java
Prettier


IngredientBusinessServiceTest.java
generated-tests > IngredientBusinessServiceTest.java > { } abc.xyz.hello.backe
Keep
Undo
1 package abc.xyz.hello.backend.businessServ
2
3 import static org.assertj.core.api.Assertions.*;
4 import static org.mockito.Mockito.*;
5
6 import abc.xyz.hello.backend.domainService.ingredient.
IIngredientService;
7 import abc.xyz.hello.backend.domainService.notification.
INotificationService;
8 import abc.xyz.hello.backend.entity.ENotificationType;
9 import abc.xyz.hello.backend.entity.Notification;
10 import abc.xyz.hello.backend.entity.UserIngredient;
11 import java.time.LocalDate;
12 import java.time.ZoneId;
13 import java.time.ZonedDateTime;
14 import java.util.ArrayList;
15 import java.util.Date;
16 import java.util.List;
17 import org.junit.jupiter.api.BeforeEach;
18 import org.junit.jupiter.api.Test;
19 import org.junit.jupiter.api.extension.ExtendWith;
20 import org.mockito.ArgumentCaptor;
21 import org.mockito.InjectMocks;
22 import org.mockito.Mock;
23 import org.mockito.junit.jupiter.MockitoExtension;
24
25 @ExtendWith(MockitoExtension.class)
26 class IngredientBusinessServ
Keep
Undo
1 of 1
28 @Mock private IIngredientService ingredientService;
CLAUDE CODE
CHAT
GENERATE UNIT TESTS FOR INGREDIENTBUSINESSSERVICE
Created IngredientBusinessServiceTest.java
Perfect! I've generated comprehensive unit tests for IngredientBusinessService and placed them in the generated-tests folder. Here's what was created:
IngredientBusinessServiceTest.java includes:
Test Coverage:
✅ PostConstruct initialization - verifies successful service initialization
✅ Happy path scenarios - single/multiple ingredients, multiple users, various expiry days
✅ Edge cases - empty lists, null inputs, mixed expiry dates
✅ Duplicate prevention - skips notifications when already sent for ingredient
✅ Exception handling - tests service exceptions and per-user error isolation
✅ Notification content - validates message format for
1 file changed +527 -1
Keep
Undo
IngredientBusinessServiceTest.java
Describe what to build
Auto
Local
Default Approvals

# Mechanisms

**CLI based** — No UI needed. Interact directly from the CLI

**Recursive BFS tracing** — follows imports transitively, not just one level deep

**Auto-detects** `src/main/java` root from the entry file path

**Classifies each file** (service, model, repo, enum, util) and groups the checklist accordingly

**Mappings are token-based** — replaces any string, so `com.classified.abc` in imports, package declarations, file names, and comments all get replaced in one pass

`mapping.json` **is portable** — copy it alongside your extracted files so you never lose the reversal key

# Reflection / Comments

- **Caveats**: you still need to look through the extracted files, though the script should already extract the necessary files to provide the necessary context for the AI to generate accurate test cases
- The generated tests on the internet will not compile and run
  (i.e. you need to bring the generated tests back into the airgap env first)
- Script currently only tested for Spring Boot project
- Important to ensure you still do not leak any confidential information out (still done manually…)
- This script could technically be used not just for generating test cases, but also for generating code for new features too